

Лабораторная работа № 3

Архитектура памяти Windows

Цель работы: получение практических навыков по использованию Win32 API для исследования памяти ОС Windows.

Задание для выполнения лабораторной работы

Разработать программу, которая:

1. выдает информацию, получаемую при использовании API GlobalMemoryStatus (при выводе информации использовать диаграммы);
2. составляет карту виртуальной памяти для любого процесса.

Программа должна выводить полную информацию о занятой и свободной физической и виртуальной памяти, составлять карту процессов, создает виртуальную память для каждого процесса.

Ход выполнения лабораторной работы

1. Создать форму с элементами для вывода информации о занятой памяти;
2. Получить информацию о занятой и свободной памяти с помощью функции GlobalMemoryStatus, которая помещает информацию в структуру;
3. После вызова функции GlobalMemoryStatus, взять необходимые данные из структуры и поместим их в соответствующие элементы на форме;
4. Создать новую форму с таблицей, в которую будет выводиться информация обо всех процессах;
5. Сделать снимок памяти с помощью функции CreateToolhelp32Snapshot, первый параметр TH32CS_SNAPPROCESS, который указывает, что нам нужен снимок именно процессов;
6. С помощью функций Process32First и Process32Next заносится информация о процессе в структуру, а из структуры, соответственно, в таблицу;
7. Получить дескриптор процесса с помощью функции OpenProcess по его ID;
8. Используя функцию GetProcessMemoryInfo получить информацию об используемой памяти выбранного процесса, которую также занести в таблицу;
9. В конце работы закрыть все открытые дескрипторы.

Код программы:

```
#include <windows.h>
#include <psapi.h>
#include <tlhelp32.h>
#include <iostream>
#include <iomanip>

void PrintMemoryInfo() {
```

```

MEMORYSTATUSEX statex;
statex.dwLength = sizeof(statex);
if (GlobalMemoryStatusEx(&statex)) {
    std::cout << "There is " << statex.dwMemoryLoad << " percent of memory in
use." << std::endl;
    std::cout << "Total physical memory: " << statex.ullTotalPhys / (1024 *
1024) << " MB" << std::endl;
    std::cout << "Free physical memory: " << statex.ullAvailPhys / (1024 *
1024) << " MB" << std::endl;
    std::cout << "Total page file: " << statex.ullTotalPageFile / (1024 *
1024) << " MB" << std::endl;
    std::cout << "Free page file: " << statex.ullAvailPageFile / (1024 *
1024) << " MB" << std::endl;
    std::cout << "Total virtual memory: " << statex.ullTotalVirtual / (1024 *
1024) << " MB" << std::endl;
    std::cout << "Free virtual memory: " << statex.ullAvailVirtual / (1024 *
1024) << " MB" << std::endl;
} else {
    std::cout << "Failed to get memory status." << std::endl;
}
}

void PrintProcessList() {
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) {
        std::cout << "Failed to take a snapshot of the processes." << std::endl;
        return;
    }

    PROCESSENTRY32 pe32;
    pe32.dwSize = sizeof(PROCESSENTRY32);
    if (Process32First(hSnapshot, &pe32)) {
        std::cout << std::left << std::setw(30) << "Process Name" <<
std::setw(10) << "PID" << std::endl;
        do {
            std::wcout << std::left << std::setw(30) << pe32.szExeFile <<
std::setw(10) << pe32.th32ProcessID << std::endl;
        } while (Process32Next(hSnapshot, &pe32));
    }
    CloseHandle(hSnapshot);
}

void PrintProcessMemoryInfo(DWORD processID) {
    HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ,
FALSE, processID);
    if (hProcess == NULL) {
        std::cout << "Failed to open process " << processID << std::endl;
        return;
    }

    PROCESS_MEMORY_COUNTERS pmc;

```

```

        if (GetProcessMemoryInfo(hProcess, &pmc, sizeof(pmc))) {
            std::cout << "Process ID: " << processID << std::endl;
            std::cout << "PageFaultCount: " << pmc.PageFaultCount << std::endl;
            std::cout << "PeakWorkingSetSize: " << pmc.PeakWorkingSetSize / 1024 << "
KB" << std::endl;
            std::cout << "WorkingSetSize: " << pmc.WorkingSetSize / 1024 << " KB" <<
std::endl;
            std::cout << "QuotaPeakPagedPoolUsage: " << pmc.QuotaPeakPagedPoolUsage /
1024 << " KB" << std::endl;
            std::cout << "QuotaPagedPoolUsage: " << pmc.QuotaPagedPoolUsage / 1024 <<
" KB" << std::endl;
            std::cout << "QuotaPeakNonPagedPoolUsage: " <<
pmc.QuotaPeakNonPagedPoolUsage / 1024 << " KB" << std::endl;
            std::cout << "QuotaNonPagedPoolUsage: " << pmc.QuotaNonPagedPoolUsage /
1024 << " KB" << std::endl;
            std::cout << "PagefileUsage: " << pmc.PagefileUsage / 1024 << " KB" <<
std::endl;
            std::cout << "PeakPagefileUsage: " << pmc.PeakPagefileUsage / 1024 << "
KB" << std::endl;
        } else {
            std::cout << "Failed to get process memory info for process ID " <<
processID << std::endl;
        }

        CloseHandle(hProcess);
    }

int main() {
    std::cout << "Memory Information:" << std::endl;
    PrintMemoryInfo();

    std::cout << "\nProcess List:" << std::endl;
    PrintProcessList();

    DWORD pid;
    std::cout << "\nEnter the process ID to get memory information, or 0 to exit:
";
    std::cin >> pid;
    if (pid != 0) {
        PrintProcessMemoryInfo(pid);
    }

    return 0;
}

```

Вывод:

```
Memory Information:
There is 92 percent of memory in use.
Total physical memory: 7834 MB
Free physical memory: 573 MB
Total page file: 24522 MB
Free page file: 4658 MB
Total virtual memory: 134217727 MB
Free virtual memory: 134213576 MB
```

```
Process List:
Process Name          PID
[System Process]      0
System                4
Secure System         108
Registry              144
smss.exe               720
csrss.exe              832
wininit.exe           1068
csrss.exe              1076
services.exe          1140
lsaiso.exe             1152
lsass.exe              1176
winlogon.exe           1236
svchost.exe            1368
WUDFHost.exe           1396
fontdrvhost.exe        1416
fontdrvhost.exe        1420
svchost.exe            1540
svchost.exe            1588
svchost.exe            1676
svchost.exe            1696
svchost.exe            1716
svchost.exe            1796
```

И т.д.

Далее можно рассмотреть конкретный процесс

```
powershell.exe        37672
WindowsDebugLauncher.exe 30820
gdb.exe                37852
c.exe                  36684

Enter the process ID to get memory information, or 0 to exit: 36684
Process ID: 36684
PageFaultCount: 1902
PeakWorkingSetSize: 4348 KB
WorkingSetSize: 3432 KB
QuotaPeakPagedPoolUsage: 27 KB
QuotaPagedPoolUsage: 27 KB
QuotaPeakNonPagedPoolUsage: 4 KB
QuotaNonPagedPoolUsage: 3 KB
PagefileUsage: 688 KB
PeakPagefileUsage: 1400 KB
```